AMBA APB

Digital VLSI Design

Actor: Hoang Bao Long

1. Specifications

Digital VLSI Design

Actor: Hoang Bao Long

1. Specifications

1.1. Overview of AMBA Protocol

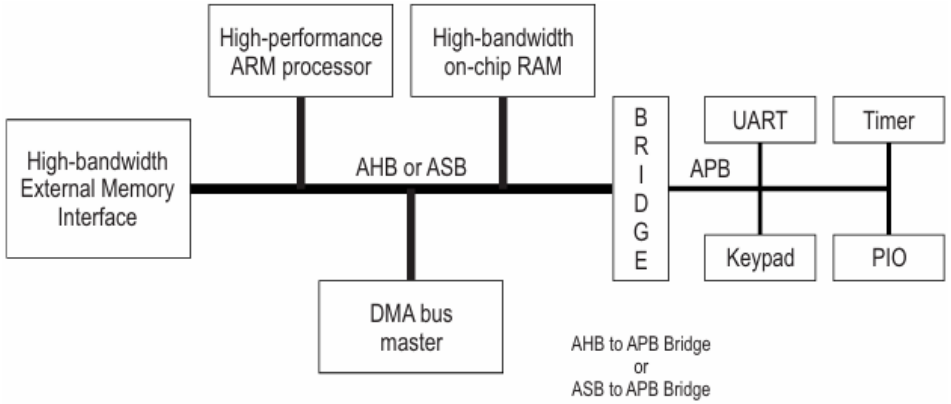
The Advanced Microcontroller Bus Architecture (AMBA) specification defines an on-chip communications standard for designing high-performance embedded microcontrollers.

Four distinct buses are defined within the AMBA specification:

- The Advanced High-performance Bus (AHB)
- The Advanced System Bus (ASB)
- The Advanced Peripheral Bus (APB).
- The Advanced eXtensible Interface (AXI)

1. Specifications

1.1. Overview of AMBA Protocol



AMBA AHB	AMBA ASB	AMBA APB
High performance	High performance	Low power
Pipelined operation	Pipelined operation	Latched address and control
Multiple bus masters	Multiple bus masters	Simple interface
Burst transfers		Suitable for many peripherals
Split transactions		

1. Specifications

1.2. Key features of the AMBA Advanced Peripheral Bus (APB)

Low-cost interface: APB is designed with a simple architecture that minimizes hardware resources and implementation cost.

Low power consumption: The non-pipelined and simple transfer mechanism helps reduce power usage, making APB suitable for peripheral devices.

Synchronous protocol: All APB transactions are synchronized to the system clock.

Non-pipelined operation: Transfers are performed sequentially without overlapping phases.

Low bandwidth, optimized for control register access
APB is intended for accessing programmable control registers rather than high-throughput data transfers.

1. Specifications

1.2. Key features of the AMBA Advanced Peripheral Bus (APB)

Designed for simple peripheral devices: Commonly used for peripherals such as GPIO, Timers, UART, SPI, and I2C.

Accessed through an APB bridge: APB peripherals are typically connected to the main system bus through a bridge (e.g., AXI-to-APB bridge).

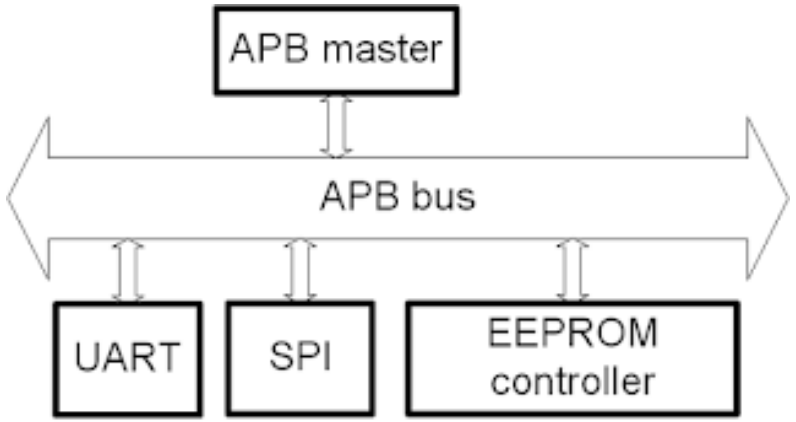
Clear Requester–Completer model:

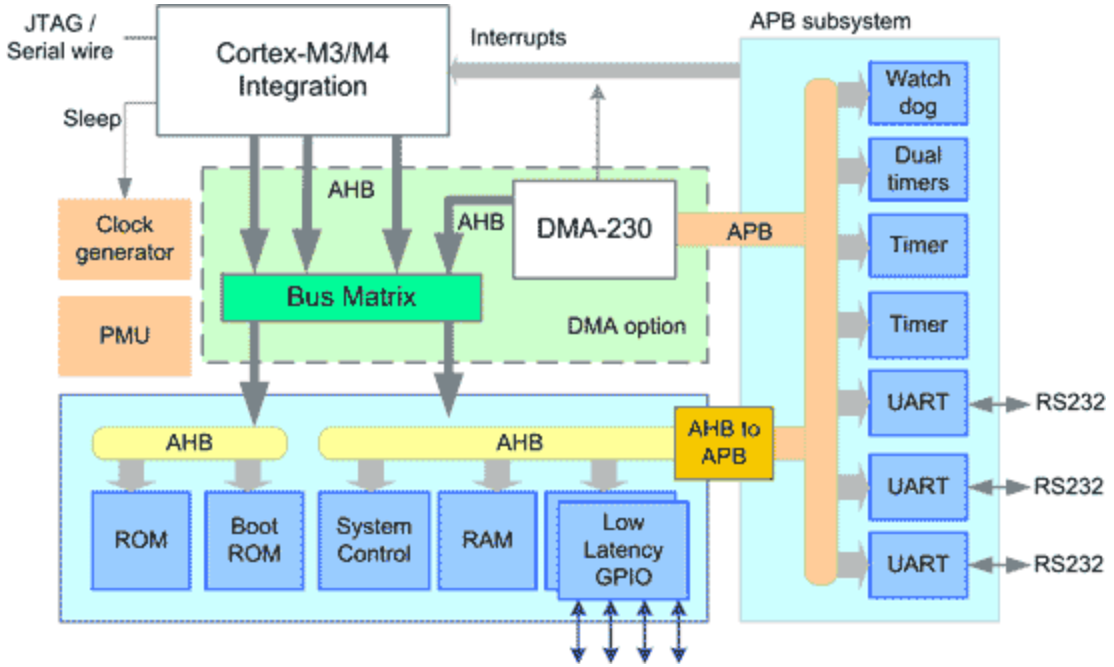
- The APB bridge acts as the Requester, initiating transactions.
- The peripheral device acts as the Completer, responding to requests.

Well-suited for AMBA-based multi-bus systems: APB integrates seamlessly with other AMBA buses such as AXI and AHB via bridges.

1. Specifications

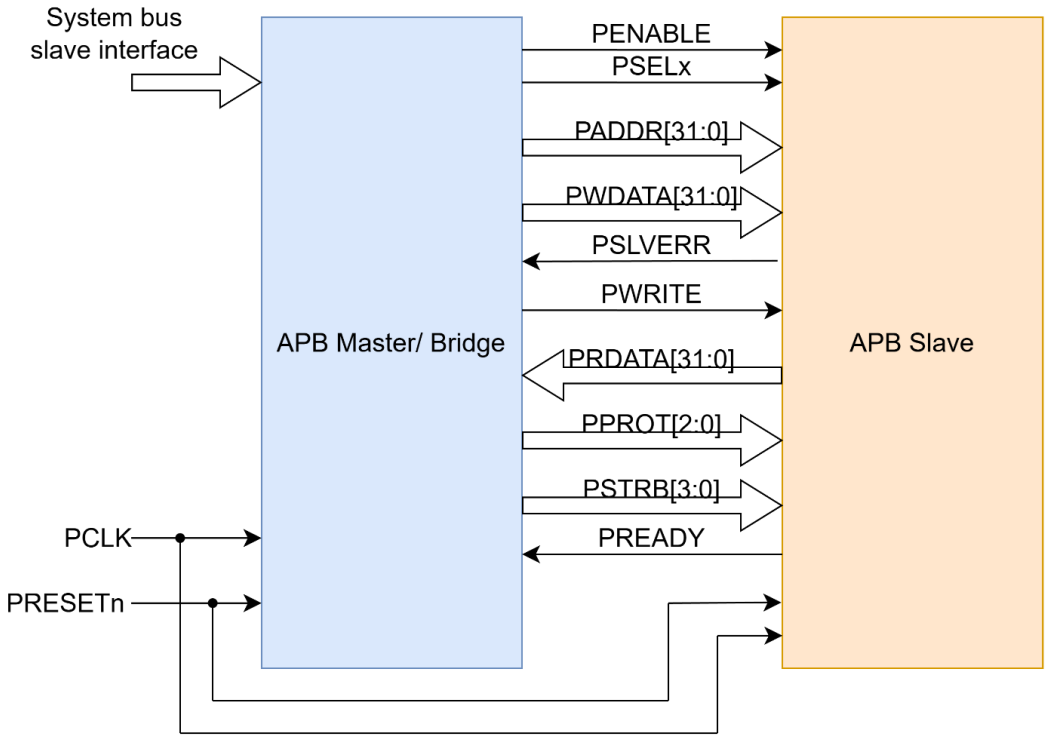
1.2. Key features of the AMBA Advanced Peripheral Bus (APB)





1. Specifications

1.3. AMBA APB signals



Block Diagram & Signal Description of APB

1. Specifications

1.3. AMBA APB signals

Signal	Source	Width	Description
PCLK	Clock Source	1	Clock. The rising edge of PCLK times all transfer on the APB
PRESETn	System Bus	1	Reset. The APB reset signal is active LOW. This signal is normally connected directly to the system bus reset signal.
PADDR	APB bridge	ADDR_WIDTH	Address. This is APB address bus. It can be up to 32 bits wide and is driven by the peripheral bus bridge unit.
PPROT	APB bridge	3	Protection type. This signal indicates the normal, privileged, or secure protection level of the transaction and whether the transaction is a data access or an instruction access.
PSELx	APB bridge	1	Slave select signal, there will be one PSEL signal for each slave connected to master.. It's an active high signal.
PENABLE	APB bridge	1	Enable. It indicates the 2nd cycle of a data transfer. It's an active high signal.

1. Specifications

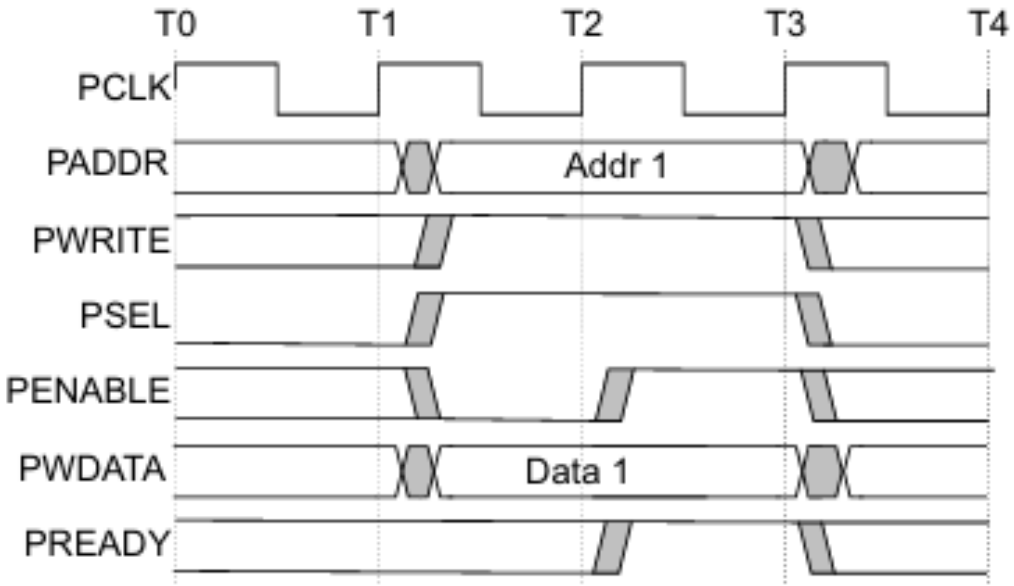
1.3. AMBA APB signals

Signal	Source	Width	Description
PWRITE	APB bridge	1	Direction. PWRITE indicates an APB write access when HIGH and an APB read access when LOW.
PWDATA	APB bridge	DATA_WIDTH	Write data. Write data bus from Master to Slave, can be up to 32 bit wide
PSTRB	APB bridge	DATA_WIDTH/8	Write strobe. PSTRB indicates which byte lanes to update during a write transfer. There is one write strobe for each 8 bits of the write data bus. PSTRB[n] corresponds to PWDATA[(8n + 7):(8n)]. PSTRB must not be active during a read transfer.
PREADY	Slave Interface	1	It is used by the slave to include wait states in the transfer. i.e. whenever slave is not ready to complete the transaction, it will request the master for some time by de-asserting the PREADY.
PSLVERR	Slave Interface	1	Indicates the Success or failure of the transfer. HIGH indicates failure and LOW indicates Success
PRDATA	Slave Interface	DATA_WIDTH	Read data us from Slave to Master, can be up to 32 bit wide

1. Specifications

1.4. APB Transfer – Write Transfers (without wait state)

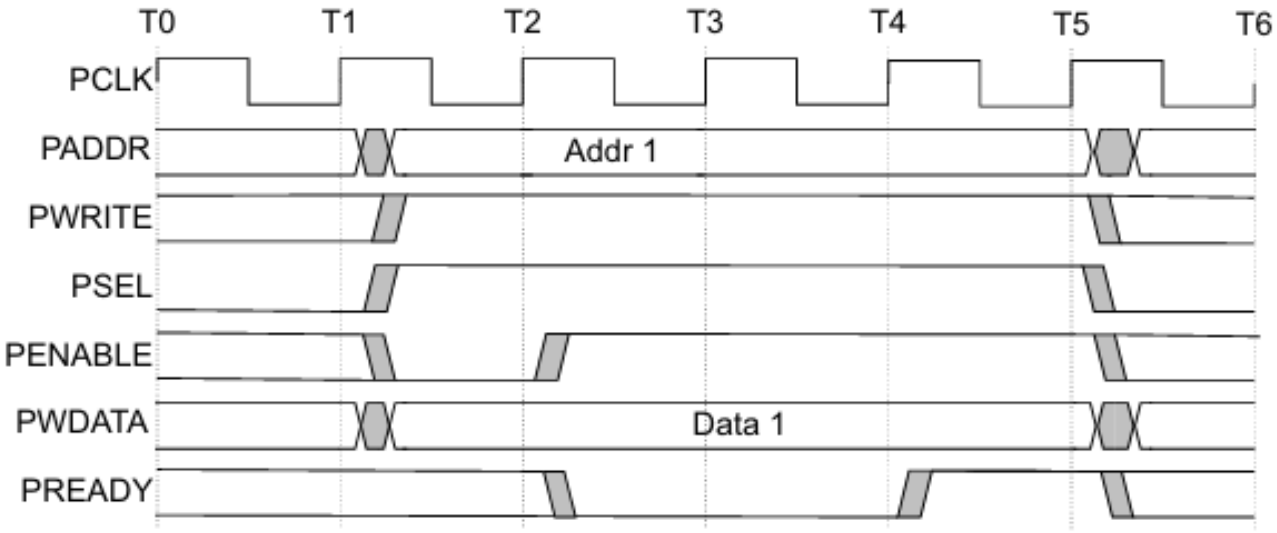
- At T1, a write transfer with address **PADDR,PWDATA,PWRITE and PSEL** starts.
- They will registered at the next rising edge of **PCLK**, T2.
- This is Setup Phase of Transfer.
- After T2, **PENABLE** and **PREADY** are registered at the rising edge of **PCLK**.
- When asserted, **PENABLE** indicates starting of ACCESS Phase
- When asserted, **PREADY** indicates that slave can complete the transfer at the next rising edige of **PCLK**.
- **PADDR, PDATA** and control signals all should remain valid till the transfer completes at T3.
- **PENABLE** signal will be de-asserted at the end of transfer.
- **PSEL** is also de-asserted, if next transfer is not to the same slave.



1. Specifications

1.4. APB Transfer – Write Transfers (with wait state)

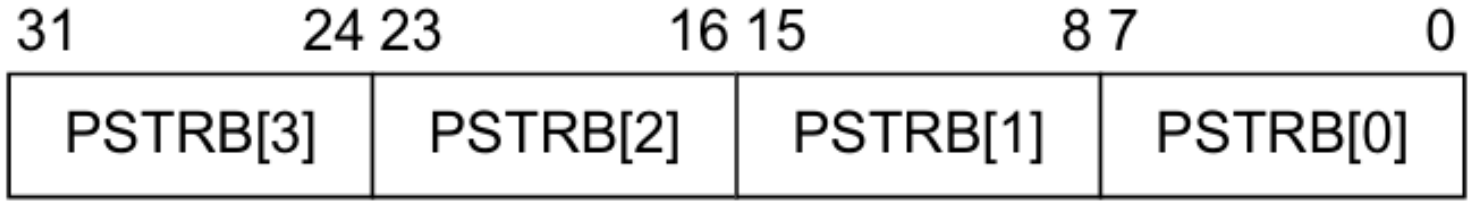
- During the ACCESS Phase, when **PENABLE** is high, the slave extends the transfer by driving **PREADY** low.
- The **PADDR, PWRITE, PSEL, PENABLE, PWDATA, PSTRB, PPROT** signals should remain unchanged while **PREADY** is low
- **PREADY** can take any value when **PENABLE** is low.
- It is recommended that the address and write signals are not changed immediately after a transfer, but remain stable until another access occurs.



1. Specifications

1.4. APB Transfer – Write strobes

- **PSTRB** enables sparse data transfer on the write data bus. Each **PSTRB** corresponds to 1 byte of the write data bus. When asserted HIGH, **PSTRB** indicates that the corresponding byte lane of the write data bus contains valid information.
- There is one write strobe for each 8 bits of the write data bus, so **PSTRB[n]** corresponds to **PWDATA[(8n + 7):(8n)].**



1. Specifications

1.4. APB Transfer – Write strobes

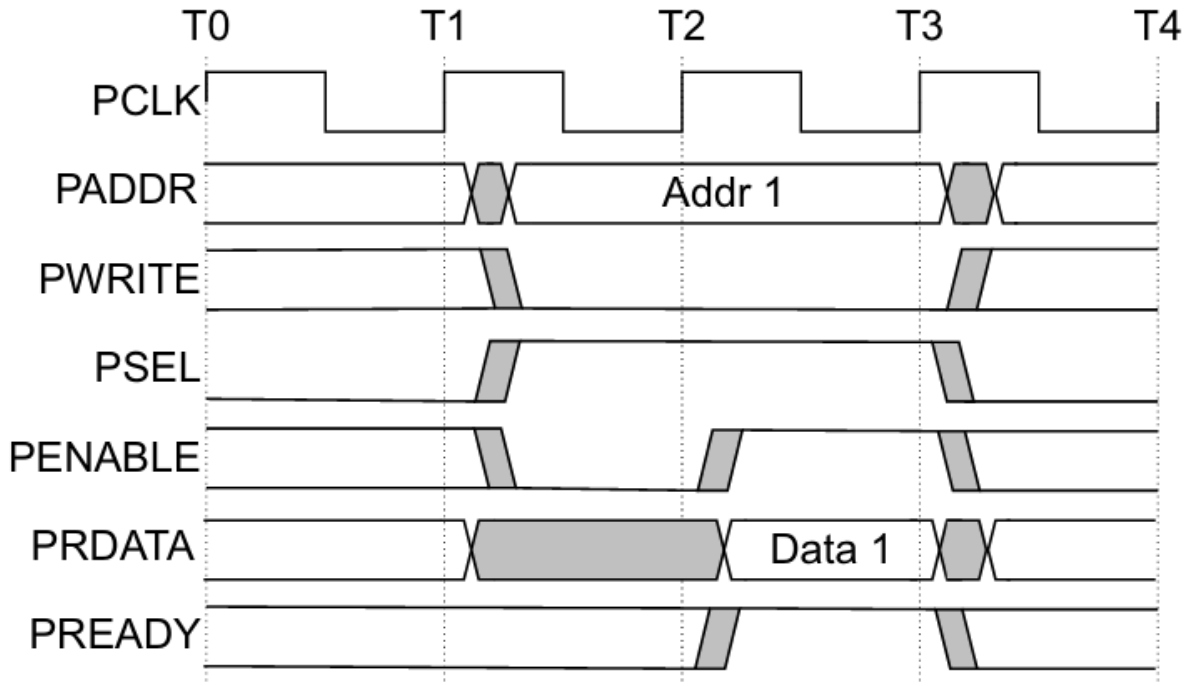
PSTRB is an optional signal. An APB peripheral might support a limited set of access types, which must be documented for the programmer. This means that all combinations of **PSTRB** presence might be compatible, if this document states that sparse writes are not supported.

PSTRB	Completer: signal not present	Completer: signal present
Requester: signal not present	Compatible. Sparse writes are not supported.	Compatible. All write data byte lanes are valid for a write. Tie the PSTRB inputs to the PWRITE output from the Requester.
Requester: signal present	Compatible. Sparse writes are not supported.	Compatible.

1. Specifications

1.4. APB Transfer – Read Transfers (without wait state)

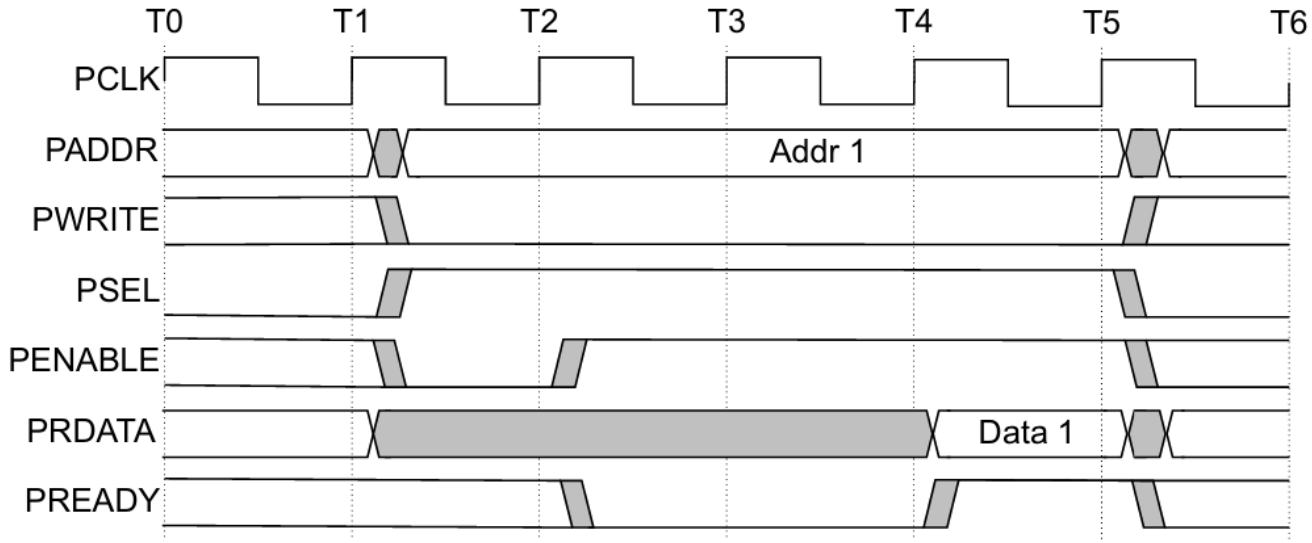
•At T1, a READ transfer with address **PADDR, PWRITE** and **PSEL** starts.
•They will be registered at rising edge of **PCLK**.
•This is SETUP Phase of the transfer.
•After T2, **PENABLE and PREADY** are registered at the rising edge of **PCLK**.
•When asserted, **PENABLE** indicates the starting of ACCESS phase.
•When asserted, **PREADY** indicates that slave can complete the transfer at next rising edge of **PCLK** by providing the data on **PRDATA**.
•Slave must provide the data before the end of read transfer. i.e. before T3.



1. Specifications

1.4. APB Transfer – Read Transfers (with wait state)

•During the ACCESS Phase, when **PENABLE** is high, the slave extends the transfer by driving **PREADY** low.
•The **PADDR, PWRITE, PSEL, PENABLE, PPROT** signals should remain unchanged while **PREADY** is low



1. Specifications

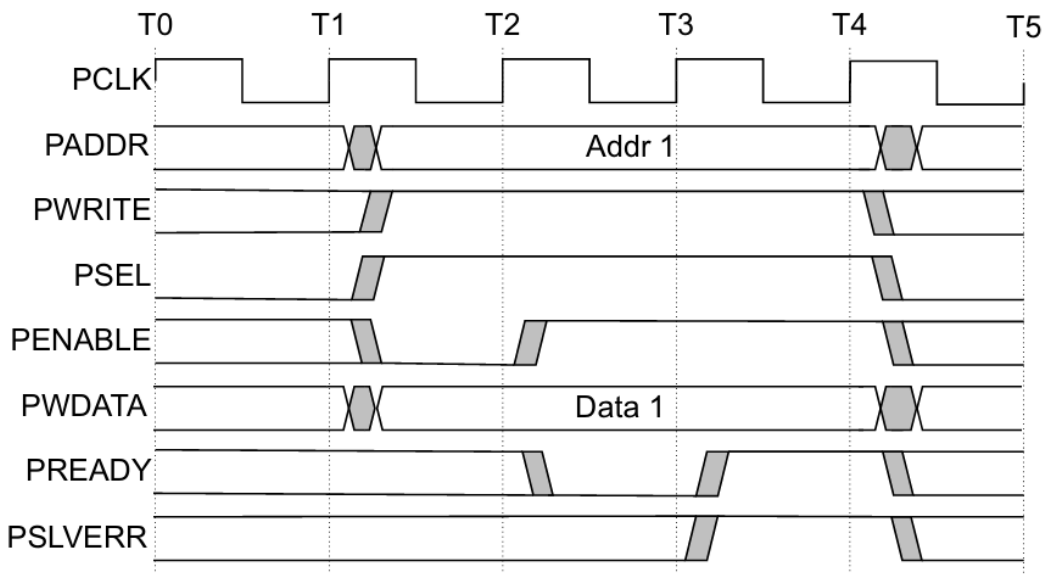
1.4. APB Transfer – Error response

Whenever there is a problem in the transfer, Slave indicates the error response for the transfer by asserting the **PSLVERR** signal. **PSLVERR** is only considered valid during the last cycle f and APB transfer, when **PSEL, PENABLE and PREADY** are all HIGH. It is recommended, but not mandatory that you drive **PSLVERR** low when it is not being sampled.

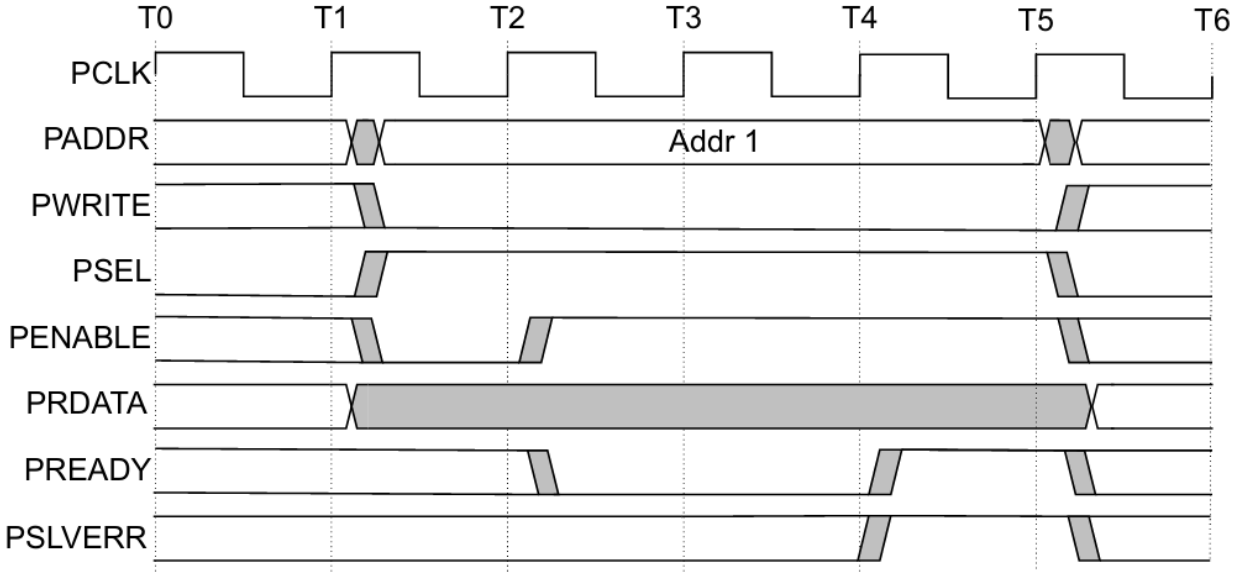
Transactions that receive an error response, might or might not have changed the state of peripheral. For example, If APB master performs a write transaction to an APB slave and received an error response, it is not guaranteed that the data is not written on the slave peripheral.

1. Specifications

1.4. APB Transfer – Error response



Write transfer



Read transfer

1. Specifications

1.4. APB Transfer – Protection until support

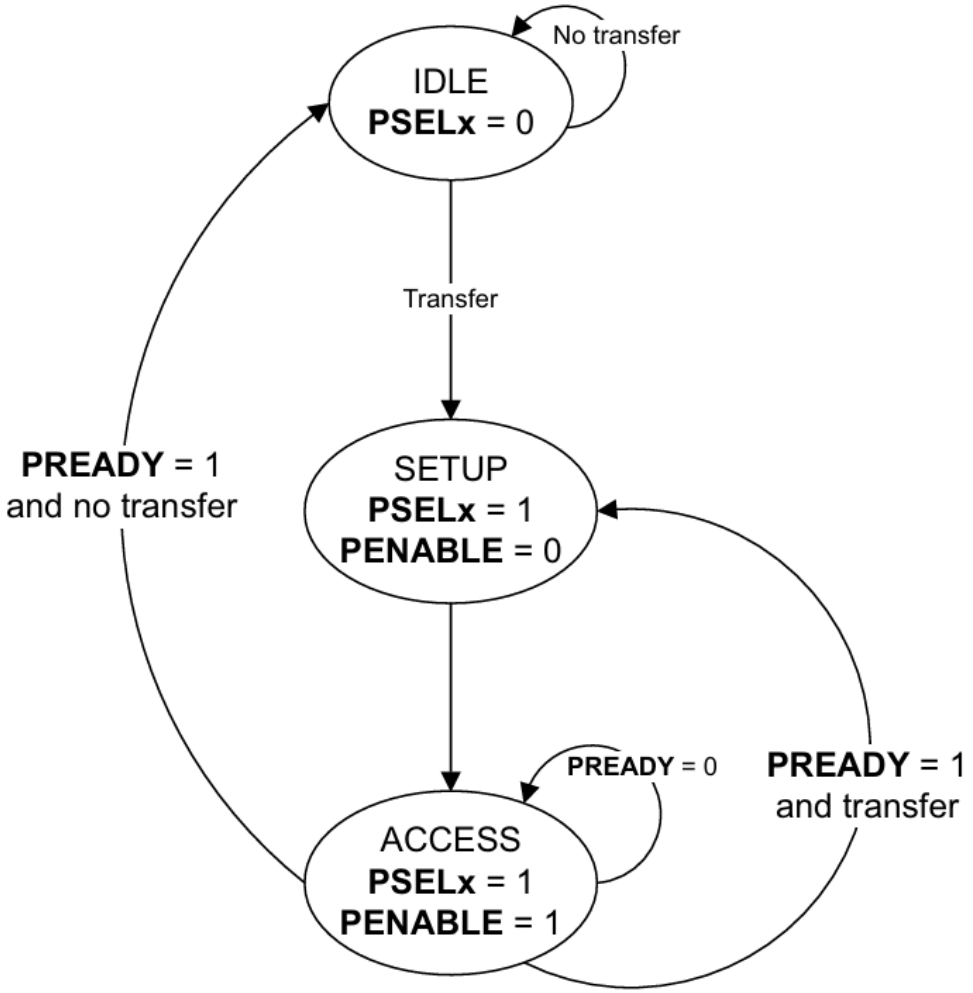
To support complex system designs, it is often necessary for both the interconnect and other devices in the system to provide protection against illegal transactions. It is provided by Protection Unit in APB Protocol. The signals indicating the protection unit are PPROT[2:0].

PPROT	Protection	Description	Comments
PPROT[0]	Normal or Privileged	PPROT[0] is used by Requesters to indicate processing mode. A privileged processing mode typically has a greater level of access within a system.	• LOW indicates normal access. • HIGH indicates privileged access.
PPROT[1]	Secure or Non-secure	PPROT[1] is used in systems where a greater degree of differentiation between processing modes is required.	• LOW indicates secure access. • HIGH indicates non-secure access
PPROT[2]	Data or Instruction	PPROT[2] gives an indication if the transaction is a data or instruction access. The transaction indication is provided as a hint and might not be accurate in all cases.	• LOW indicates data access. • HIGH indicates instruction access.

1. Specifications

1.4. APB Operation States

IDLE	This is the default state of the APB interface.
SETUP	When a transfer is required, the interface moves into the SETUP state, where the appropriate select signal, PSELx, is asserted. The interface only remains in the SETUP state for one clock cycle and always moves to the ACCESS state on the next rising edge of the clock.

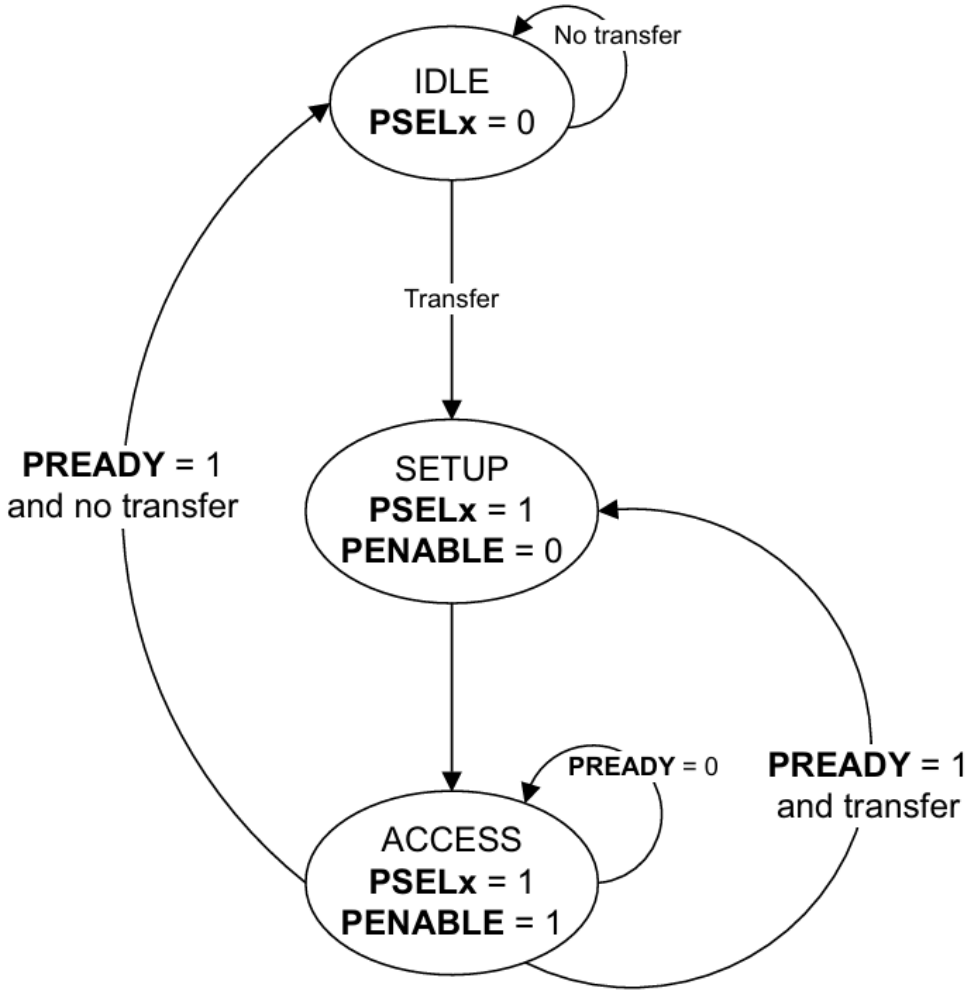


1. Specifications

1.4. APB Operation States

ACCESS

- **PENABLE** is asserted in the ACCESS state. During the transition from SETUP to ACCESS and throughout the ACCESS state, the following signals must remain stable: ***PADDR, PPROT, PWRITE, PSTRB, PAUSER, PWUSER, and PWDATA*** (for write transfers only).
- The exit from the ACCESS state depends on **PREADY** from the Completer: if **PREADY** is LOW, the bus ***stays in*** **ACCESS**; if **PREADY** is HIGH, the bus ***leaves*** **ACCESS** and returns to IDLE or moves directly to SETUP for the next transfer.



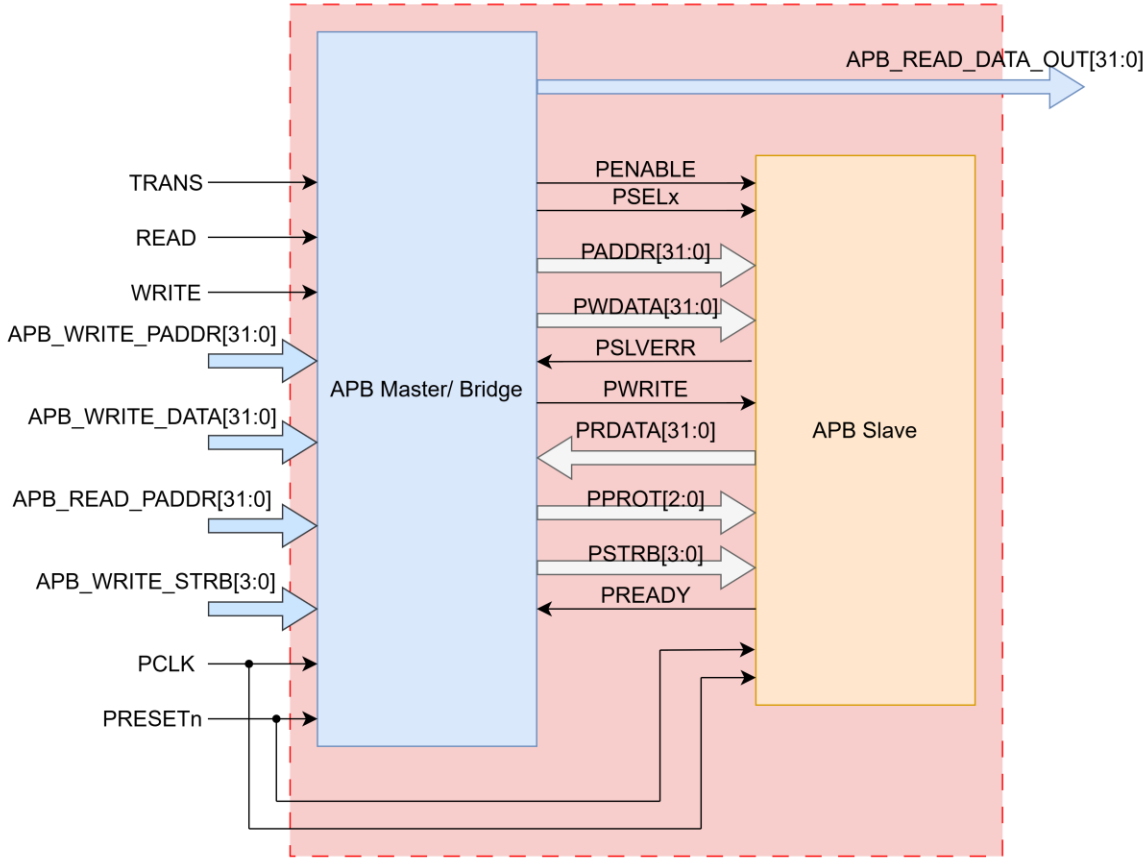
2. Design Specifications

Digital VLSI Design

Actor: Hoang Bao Long

2. Design Specifications

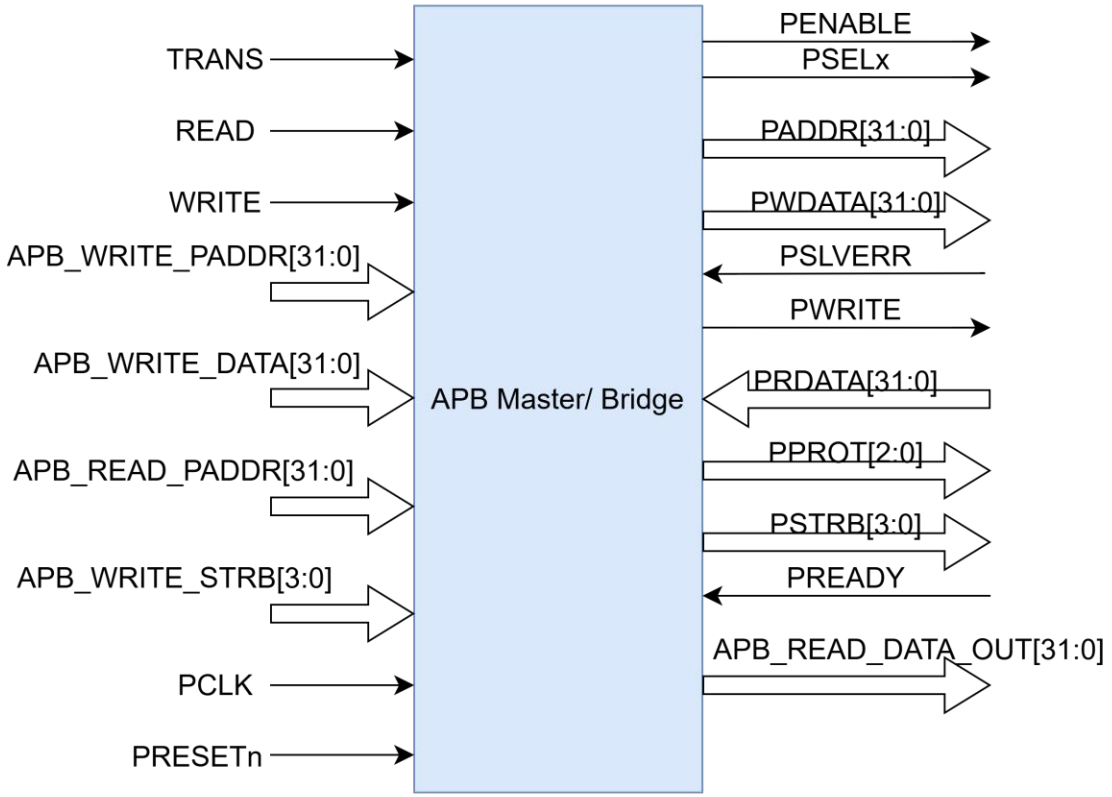
2.1. APB Wrapper



APB Interface

2. Design Specifications

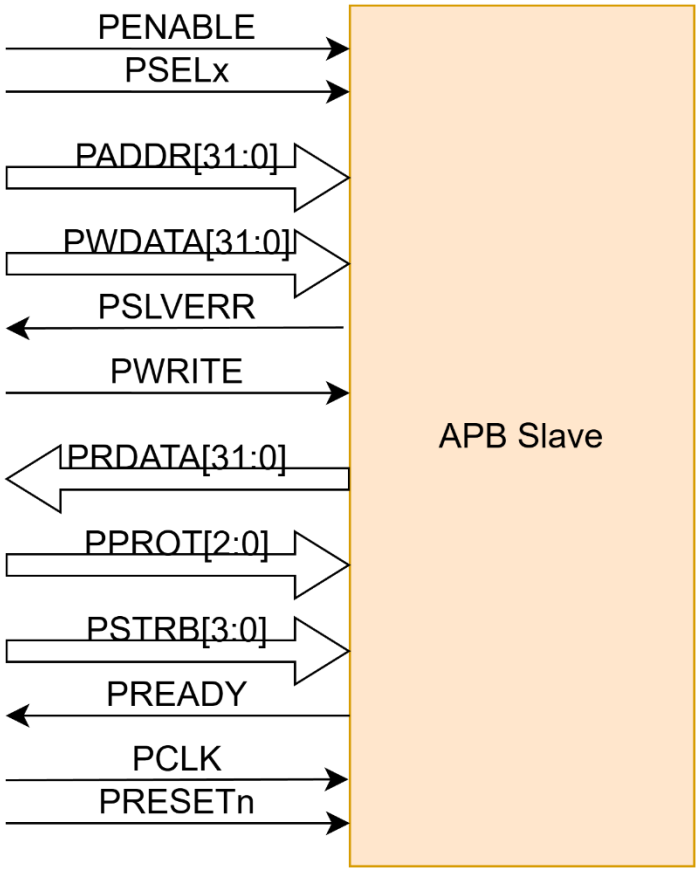
2.2. APB Master



APB Master block

2. Design Specifications

2.3. APB Slave



APB Slave block

2. Design Specifications

2.4. APB Signals

Signal	Source	Width	Description
TRANS	System BUS	1	Start signal. When the TRANS signal is set to HIGH, the system will begin operation. This signal may remain active for one clock cycle or multiple cycles.
READ	System BUS	1	Read signal.
WRITE	System BUS	1	Write signal.
APB_WRITE_PADDR	System BUS	ADDR_WIDTH	Address. This is the APB write address used by the APB master to write data to the slave. The address can be up to 32 bits wide.
APB_WRITE_DATA	System BUS	DATA_WIDTH	Data Write. This is the data sent from the APB master to the APB slave during a write operation. The data width can be up to 32 bits.
APB_READ_PADDR	System BUS	ADDR_WIDTH	Address. This is the APB read address used by the APB master to read data from the slave. The address can be up to 32 bits wide.
APB_READ_DATA_OUT	System BUS	DATA_WIDTH	Data Out. This is the data which is read from slave. The data width can be up to 32 bits.

2. Design Specifications

2.4. APB Signals

Signal	Source	Width	Description
TRANS	System BUS	1	Start signal. When the TRANS signal is set to HIGH, the system will begin operation. This signal may remain active for one clock cycle or multiple cycles.
READ	System BUS	1	Read signal.
WRITE	System BUS	1	Write signal.
APB_WRITE_PADDR	System BUS	ADDR_WIDTH	Address. This is the APB write address used by the APB master to write data to the slave. The address can be up to 32 bits wide.
APB_WRITE_DATA	System BUS	DATA_WIDTH	Data Write. This is the data sent from the APB master to the APB slave during a write operation. The data width can be up to 32 bits.
APB_READ_PADDR	System BUS	ADDR_WIDTH	Address. This is the APB read address used by the APB master to read data from the slave. The address can be up to 32 bits wide.
APB_READ_DATA_OUT	System BUS	DATA_WIDTH	Data Out. This is the data which is read from slave. The data width can be up to 32 bits.

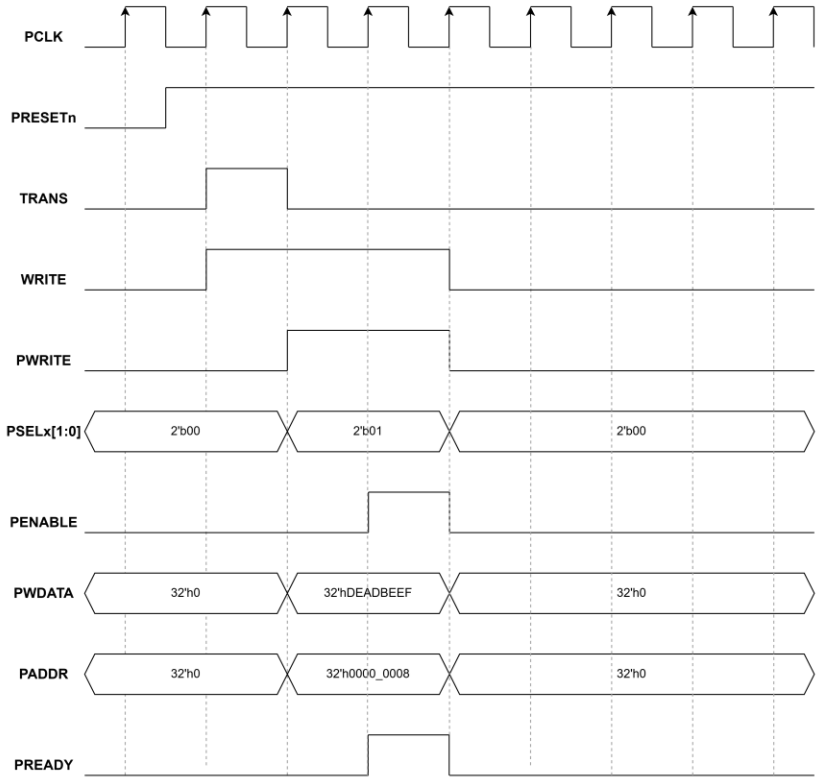
2. Design Specifications

2.4. APB Signals

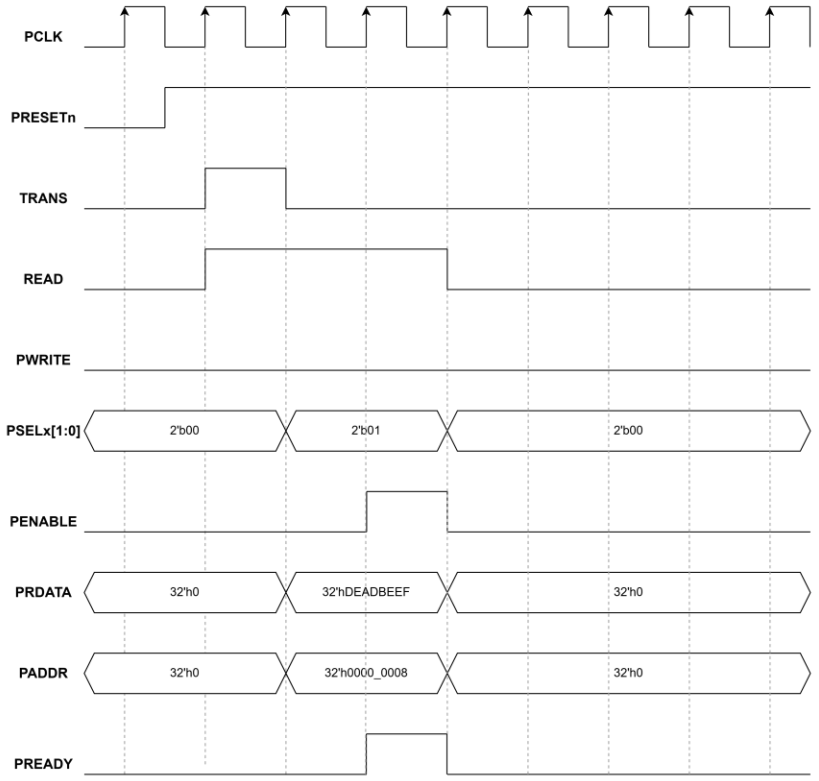
Signal	Source	Width	Description
PWRITE	APB bridge	1	Direction. PWRITE indicates an APB write access when HIGH and an APB read access when LOW.
PWDATA	APB bridge	DATA_WIDTH	Write data. Write data bus from Master to Slave, can be up to 32 bit wide
PSTRB	APB bridge	DATA_WIDTH/8	Write strobe. PSTRB indicates which byte lanes to update during a write transfer. There is one write strobe for each 8 bits of the write data bus. PSTRB[n] corresponds to PWDATA[(8n + 7):(8n)]. PSTRB must not be active during a read transfer.
PREADY	Slave Interface	1	It is used by the slave to include wait states in the transfer. i.e. whenever slave is not ready to complete the transaction, it will request the master for some time by de-asserting the PREADY.
PSLVERR	Slave Interface	1	Indicates the Success or failure of the transfer. HIGH indicates failure and LOW indicates Success
PRDATA	Slave Interface	DATA_WIDTH	Read data us from Slave to Master, can be up to 32 bit wide

2. Design Specifications

2.5. APB Operation waveform



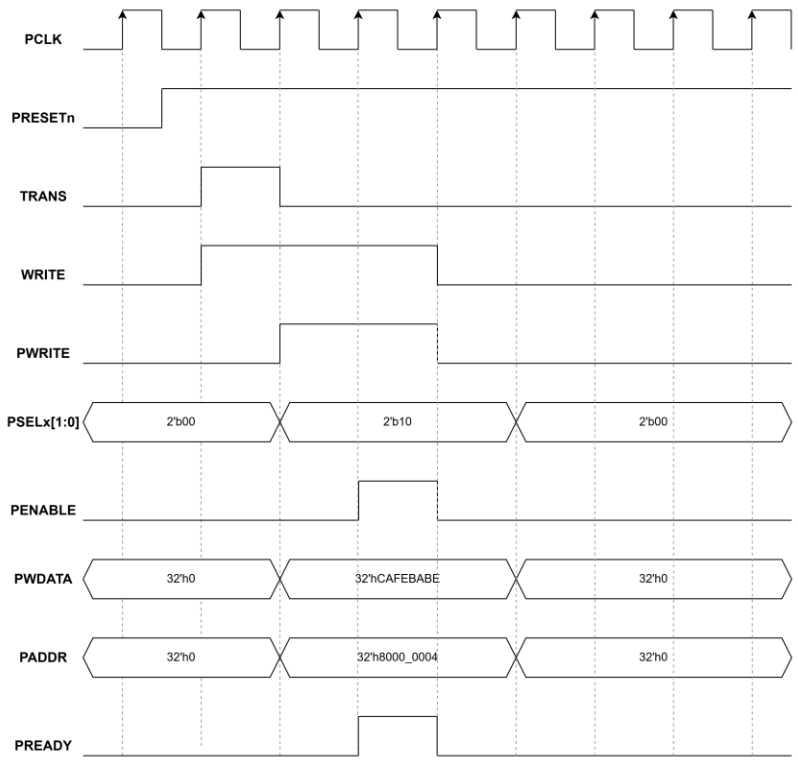
APB Write transfer without waiting state



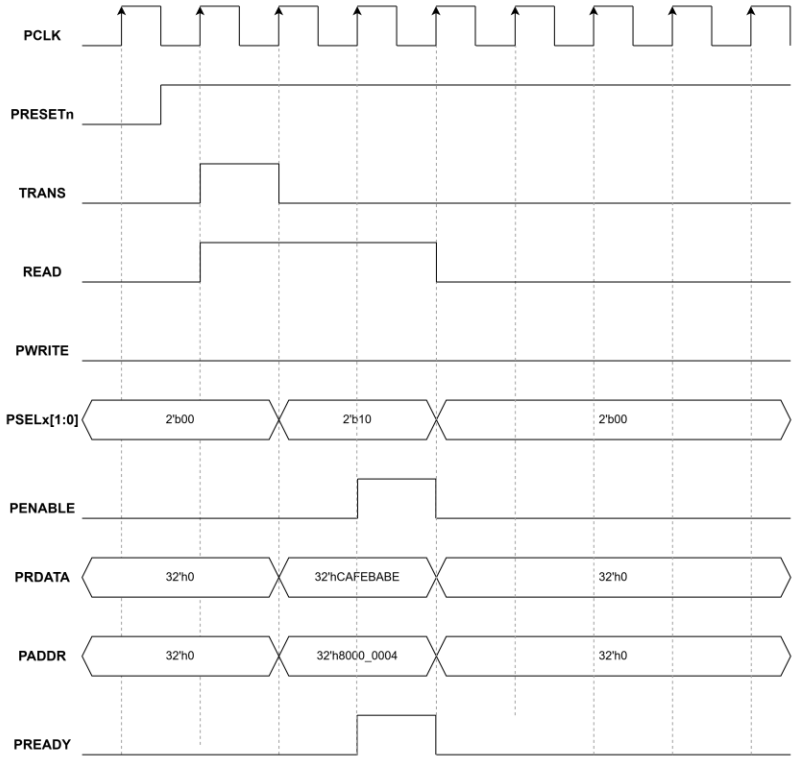
APB Read transfer without waiting state

2. Design Specifications

2.5. APB Operation waveform



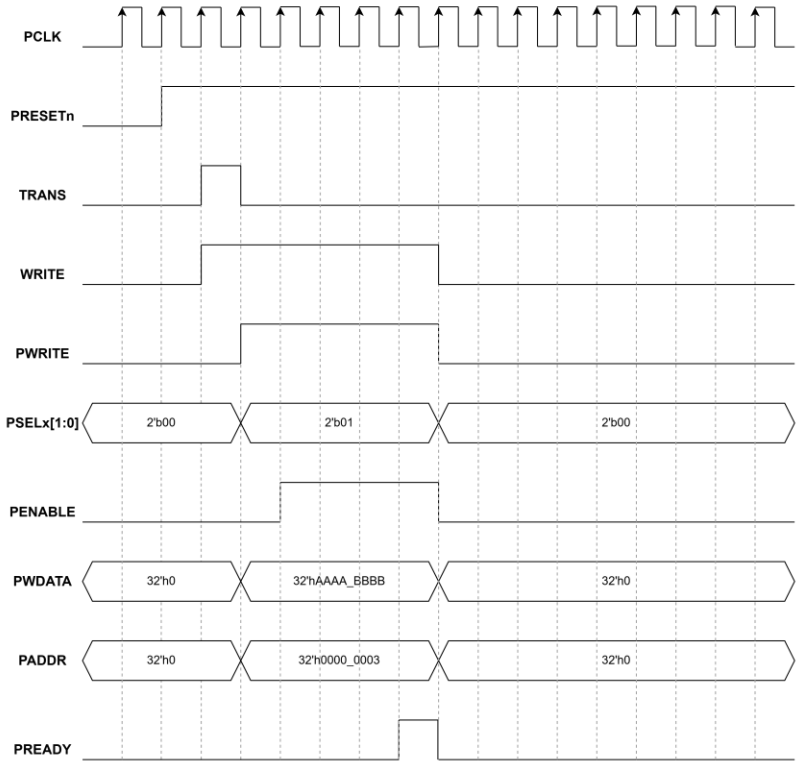
APB Write with Slave 1 access



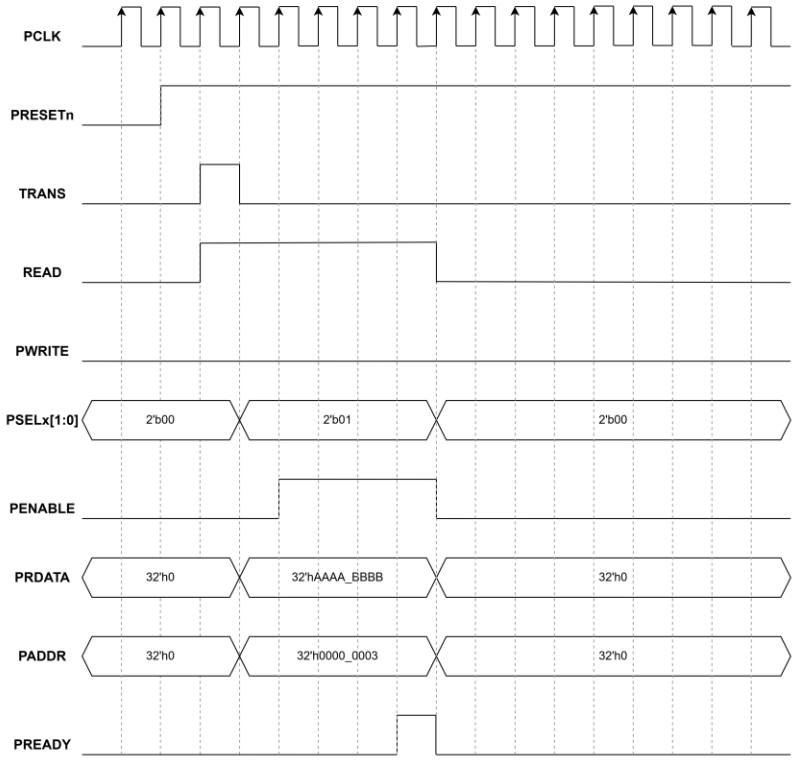
APB Read with Slave 1 access

2. Design Specifications

2.5. APB Operation waveform



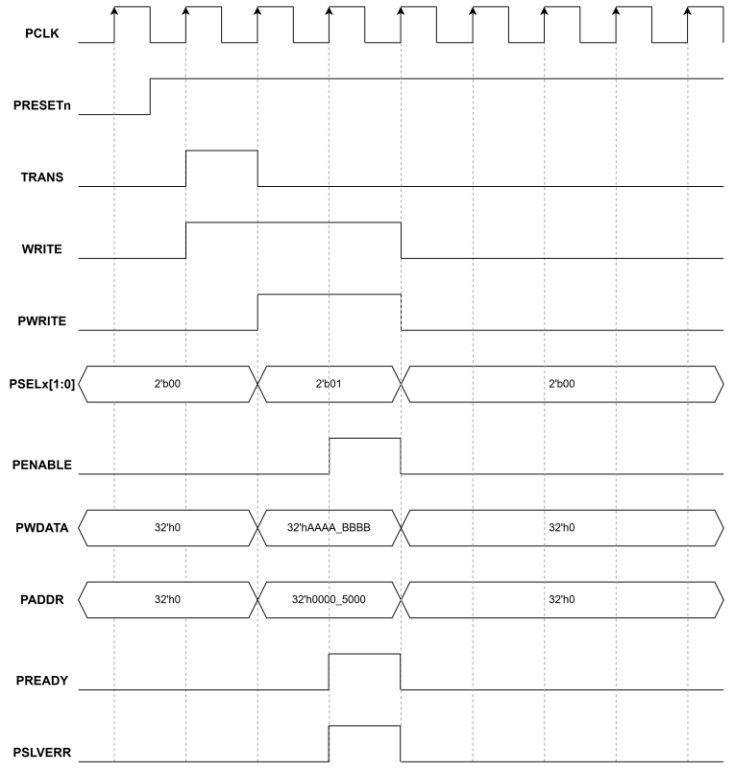
APB Write transfer with waiting state



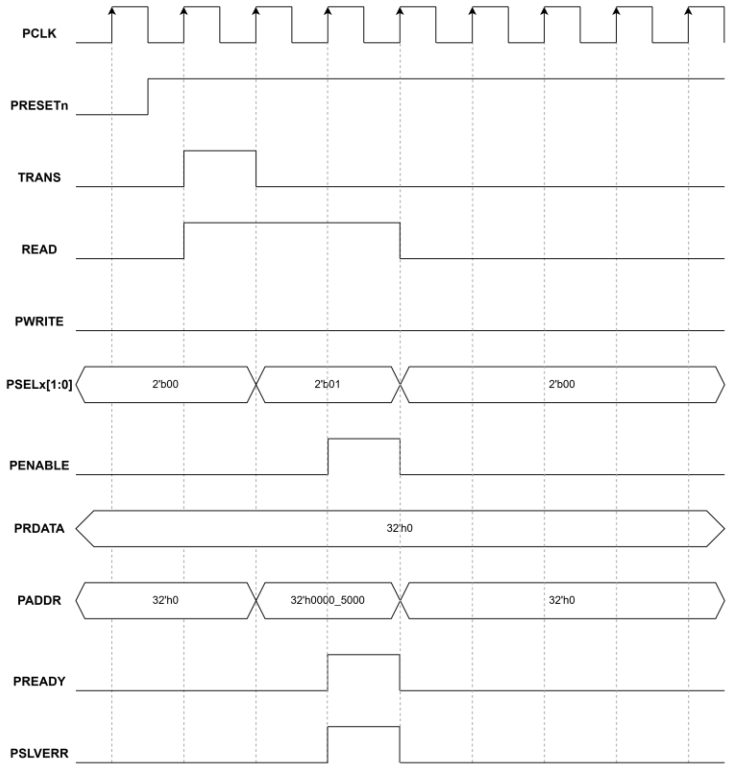
APB Read transfer with waiting state

2. Design Specifications

2.5. APB Operation waveform



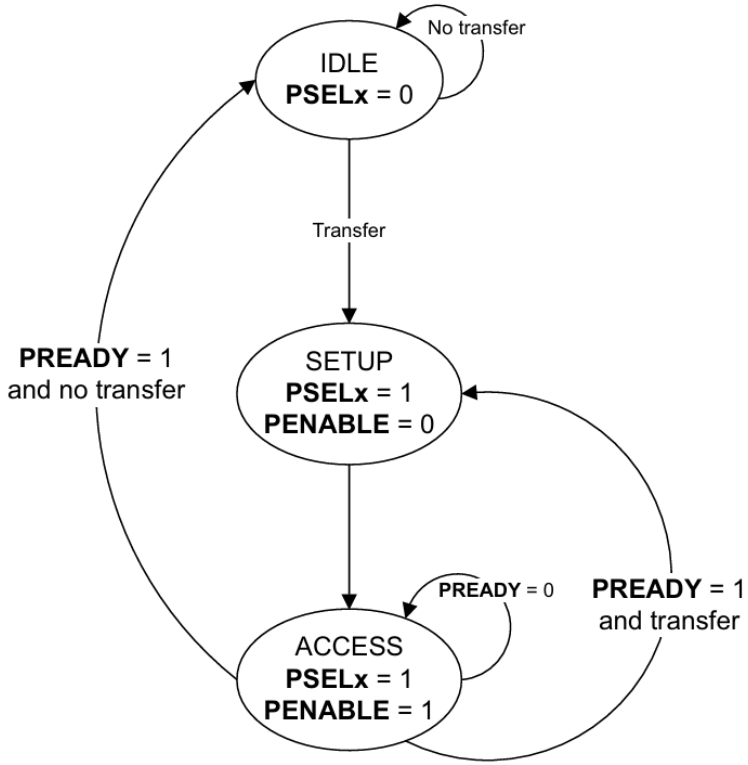
APB Write with address range error



APB Read with address range error

2. Design Specifications

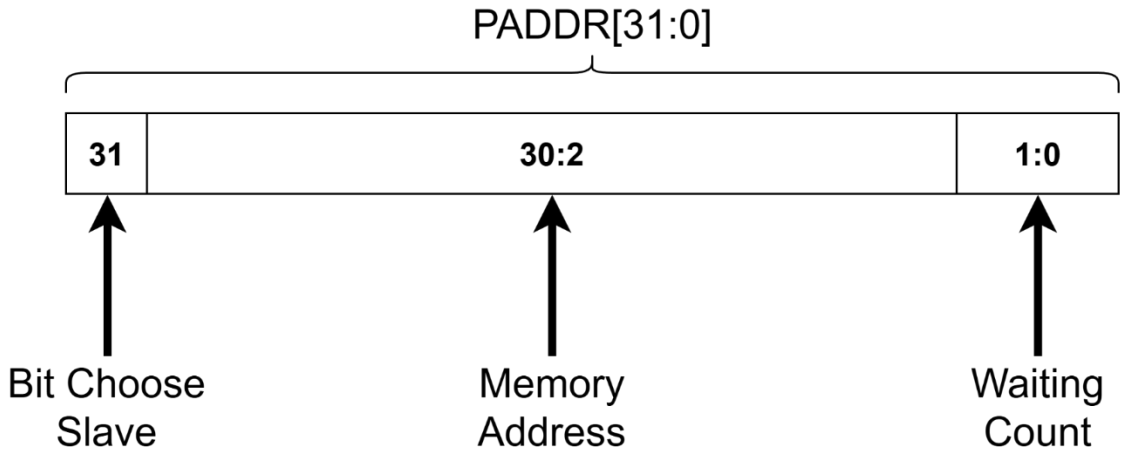
2.5. Design Spec – APB Master



APB Master FSM

2. Design Specifications

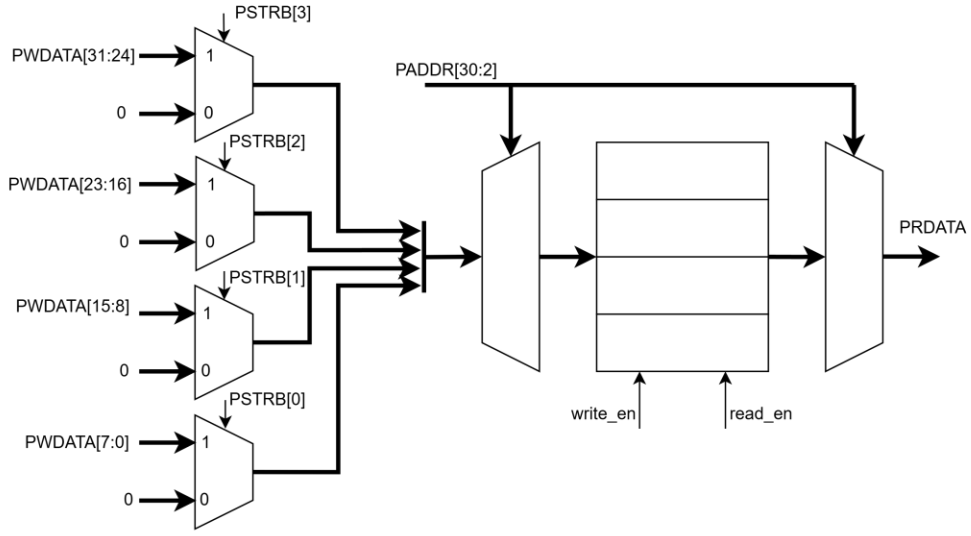
2.5. Design Spec – APB Slave



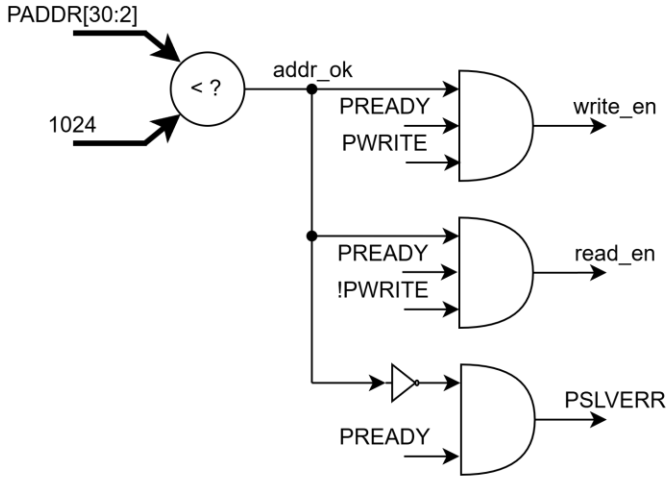
BUS Signals	Name	Descriptions
PADDR[31]	Slave Select Bit	Used to select the target slave. 0 selects Slave 0; 1 selects Slave 1.
PADDR[30:2]	Memory Address	These bits represent the internal memory index (Word-aligned). They are used to access specific data entries within the slave's memory array.
PADDR[1:0]	Waiting Count	These bits determine the number of wait states (delay cycles) the Slave will force the Master to wait before completing the transaction.

2. Design Specifications

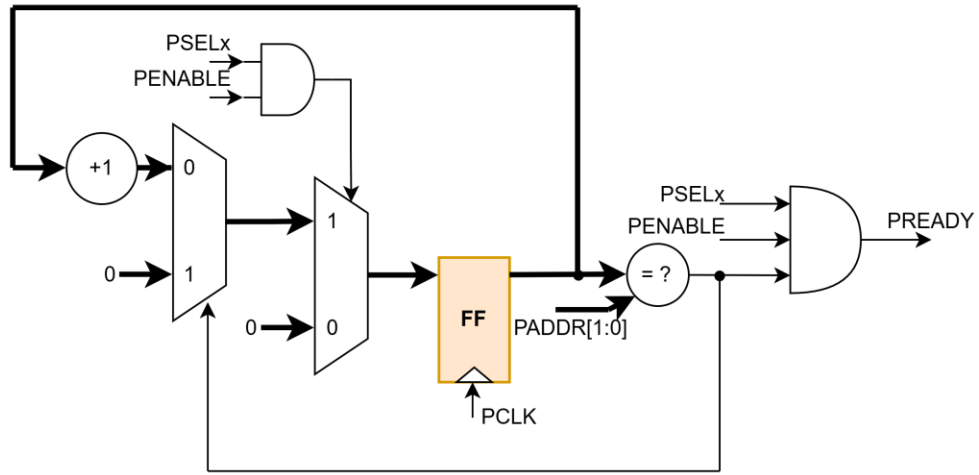
2.5. Design Spec – APB Slave



APB Slave Memory



APB Signal Controller



APB Waiting State Controller

3. Design Verification

Digital VLSI Design

Actor: Hoang Bao Long

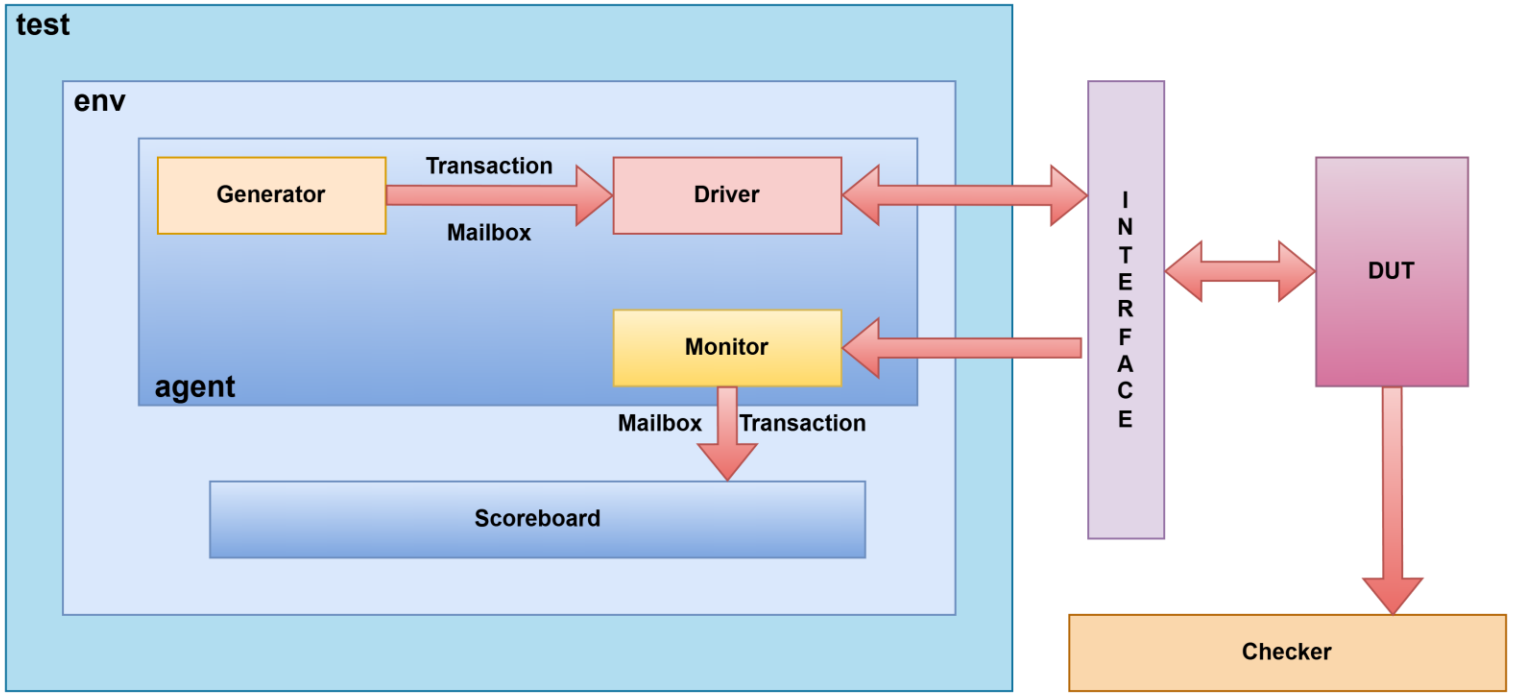
3. Design Verification

3.1. Verification Plans

Index	Priority	Feature	Sub-Feature1	Sub-Feature2	Sub-Feature3	Feature Descriptions	Test description	Pass-fail Criteria /Check point	TestPlan Input (Design Spec/CKT Spec/uArch/CSR)	Owner	Due date	status	Testname
1	High	APB Basic Write/Read	Address Decode	Fixed Address Access	Slave0	Verify APB transaction at fixed valid address	Generate transactions with address 0x0000_0004 and PSEL=01	All transfers complete successfully, no protocol violation, data write/read matched	APB Spec Ch3.1, Ch3.3, Register Map	Long		PASS	FIXED_ADDR
2	High	APB Random Access	Address Generation	Random Address	Slave Selection	Verify random APB accesses within configurable address range	Randomize address between addr_lo and addr_hi using both slaves	Address always inside configured range, correct slave selected, transaction completes	Address Decode Spec, APB Ch2.1	Long		PASS	RAND_ADDR
3	High	APB Legal Address Coverage	Valid Address Space	Multi-Slave Access	Read/Write Mix	Verify all legal register locations can be accessed	Randomly generate accesses among predefined valid registers	behavior at every valid address	Register Spec, APB Ch3	Long		PASS	RAND_ADDR_INRANGE
4	High	APB Slave0 Access	Decode Logic	Slave0 Select	Address Routing	Verify bridge selects Slave0 correctly	Transactions targeting Slave0 address map	PSEL0 asserted only, PSEL1 deasserted	Address Decoder Design	Long		PASS	SLAVE_0
5	High	APB Slave1 Access	Decode Logic	Slave1 Select	Address Routing	Verify bridge selects Slave1 correctly	Generate accesses to 0x8000_0014 with PSEL=10	PSEL1 asserted only, access reaches Slave1	Address Decoder Design	Long		PASS	SLAVE_1
6	High	APB Write Transfer	Write Transaction	No Wait State	Data Integrity	Verify write transfer execution	Generate write transactions with random PWDATA	PADDR/PWDATA stable in Setup and Access phase, write data stored correctly	APB Ch3.1.1	Long		PASS	WRITE_BASIC
7	High	APB Read Transfer	Read Transaction	No Wait State	Data Integrity	Verify read transfer execution	Generate read transactions after writes	Read data equals expected value	APB Ch3.3.1	Long		PASS	READ_BASIC
8	High	APB Wait State	Write Wait State	PREADY Extension	Stable Signals	Verify write transfer during wait state	Access odd/non-aligned waiting-state addresses from testcase	While PREADY=0, PADDR/PWDATA/PWRITE/PSEL/PENABLE remain unchanged	APB Ch3.1.2, Ch4.1	Long		PASS	WRITE_WAITING_STATE
9	High	APB Wait State	Read Wait State	PREADY Extension	Stable Signals	Verify read transfer during wait state	Read using wait-state addresses	While PREADY=0, control signals remain stable and data returns correctly	APB Ch3.3.2, Ch4.1	Long		PASS	READ_WAITING_STATE
10	High	APB Error Handling	Invalid Address	PSLVERR Generation	Error Completion	Verify invalid address response	Access address 0x0000_5000	PSLVERR asserted at final transfer cycle, transaction completed with error	APB Ch3.4	Long		PASS	ERROR_RESP
11	High	APB State Machine	IDLE→SETUP→ACCESS	ACCESS Extension	ACCESS Exit	Verify APB FSM transitions	Monitor protocol states for all testcases	State transitions follow APB specification	APB Ch4.1	Long		PASS	APB_FSM
12	High	APB Protocol Compliand	Signal Stability	Setup/Access Phase	Handshake Validation	Verify APB protocol requirements	Run all tests with protocol assertions enabled	No APB assertion failure, no signal change during ACCESS wait cycles	APB Ch3, Ch4, Appendix A	Long		PASS	PROTOCOL_CHECK

3. Design Verification

3.2. Testbench environment



3. Design Verification

3.2. Testbench environment

```
package testcase_pkg;

typedef enum logic [3:0] {
  FIXED_ADDR          = 4'b0000,
  RAND_ADDR           = 4'b0001,
  RAND_ADDR_INRANGE   = 4'b0010,
  SLAVE_0             = 4'b0011,
  SLAVE_1             = 4'b0100,
  ERROR_RESP          = 4'b0101,
  READ_WAITING_STATE  = 4'b0110,
  WRITE_WAITING_STATE = 4'b0111
} test_case;

endpackage: testcase_pkg
```

```
constraint c_psel_depends_on_addr_msb {
  PSELx == (PADDR[BUS_WIDTH-1] ? 2'b10 : 2'b01);
}
```

3. Design Verification

3.2. Testbench environment

```
covergroup cov_pwrite;
  cp_pwrite: coverpoint PWRITE{
    bins write = {1'b1};
    bins read = {1'b0};
  }
endgroup
```

```
covergroup cov_slv_apb;
  cp_paddr: coverpoint PADDR {
    bins fixed_addr         = {32'h0000_0004};
    bins rand_addr_in_range = {32'h0000_0004,
                               32'h8000_0004,
                               32'h0000_0008,
                               32'h0000_0020,
                               32'h8000_0010};
    bins addr_error  = {32'h0000_5000};
    bins addr_slave1 = {32'h8000_0014};
    bins waiting_addr_slv0 = { 32'h0000_0001, 32'h0000_0002, 32'h0000_0006 };
    bins waiting_addr_slv1 = { 32'h8000_0001, 32'h8000_0002, 32'h8000_0006 };
  }
  cp_pselx: coverpoint PSELx {
    bins slave_0 = {2'b01};
    bins slave_1 = {2'b10};
  }
  cross_paddr_pselx : cross cp_paddr, cp_pselx {
  bins addr_slv0 = binsof(cp_paddr.fixed_addr) && binsof(cp_pselx.slave_0);
  bins addr_slv1 = binsof(cp_paddr.addr_slave1) && binsof(cp_pselx.slave_1);
  bins waiting_slv0 = binsof(cp_paddr.waiting_addr_slv0) && binsof(cp_pselx.slave_0);
  bins waiting_slv1 = binsof(cp_paddr.waiting_addr_slv1) && binsof(cp_pselx.slave_1);
  ignore_bins false_addr_slv0 =
      binsof(cp_paddr.fixed_addr) &&
      binsof(cp_pselx.slave_1) ||
      binsof(cp_paddr.waiting_addr_slv0) &&
      binsof(cp_pselx.slave_1);
  ignore_bins false_addr_slv1 =
      binsof(cp_paddr.addr_slave1) &&
      binsof(cp_pselx.slave_0) ||
      binsof(cp_paddr.waiting_addr_slv1) &&
      binsof(cp_pselx.slave_0);
  ignore_bins error_addr0 = binsof(cp_paddr.addr_error) && binsof(cp_pselx.slave_0);
  ignore_bins error_addr1 = binsof(cp_paddr.addr_error) && binsof(cp_pselx.slave_1);
}
endgroup
```

4. Verification Results

Digital VLSI Design

Actor: Hoang Bao Long

4. Verification Results

AHB Regression Report

Generated : 2026-07-16 09:29:17

Regression Summary

Test	Status	Writes	Reads	SB Errors	Warnings
ERROR_RESP	**PASS**	7	3	0	0
FIXED_ADDR	**PASS**	3	5	0	0
RAND_ADDR	**PASS**	9	7	0	0
RAND_ADDR_INRANGE	**PASS**	17	13	0	0
READ_WAITING_STATE	**PASS**	6	14	0	0
SLAVE_0	**PASS**	5	5	0	0
SLAVE_1	**PASS**	1	9	0	0
WRITE_WAITING_STATE	**PASS**	16	4	0	0

4. Verification Results

Assertion Analysis

Test	SVA PASS	SVA FAIL	SVA WARN	COVER
ERROR_RESP	50	0	0	10
FIXED_ADDR	32	0	0	8
RAND_ADDR	64	0	0	16
RAND_ADDR_INRANGE	120	0	0	30
READ_WAITING_STATE	80	0	0	20
SLAVE_0	40	0	0	10
SLAVE_1	40	0	0	10
WRITE_WAITING_STATE	80	0	0	20

Failed Assertion Summary

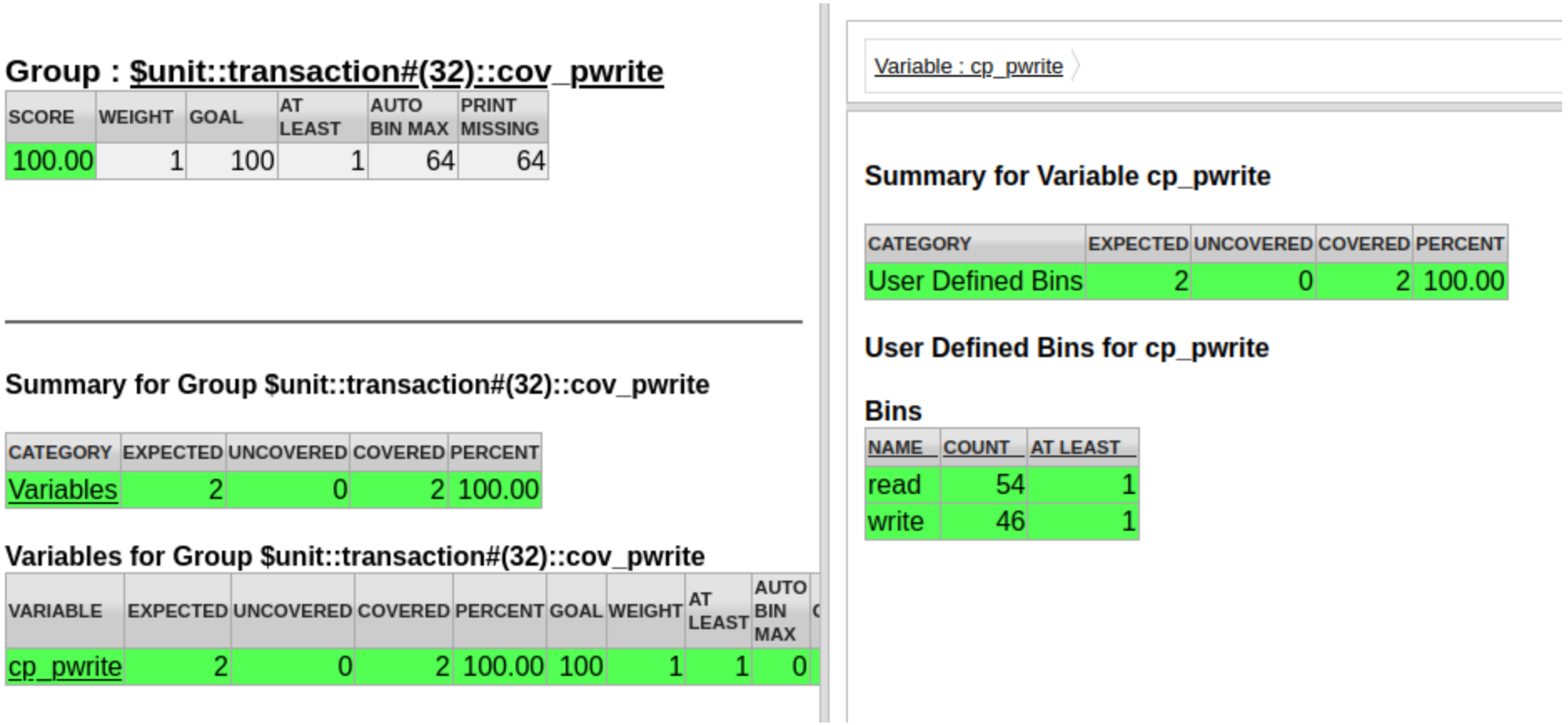
Assertion	Fail Count
No Assertion Failure	

4. Verification Results

Overall Statistics

Total Tests	PASS	FAIL	Total SVA PASS	Total SVA FAIL	Total SVA WARN
8	8	0	506	0	0

4. Verification Results



Group : $unit::transaction#(32)::cov_pwrite

SCORE	WEIGHT	GOAL	AT LEAST	AUTO BIN MAX	PRINT MISSING
100.00	1	100	1	64	64

Summary for Group $unit::transaction#(32)::cov_pwrite

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
Variables	2	0	2	100.00

Variables for Group $unit::transaction#(32)::cov_pwrite

VARIABLE	EXPECTED	UNCOVERED	COVERED	PERCENT	GOAL	WEIGHT	AT LEAST	AUTO BIN MAX
cp_pwrite	2	0	2	100.00	100	1	1	0

Variable : cp_pwrite

Summary for Variable cp_pwrite

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
User Defined Bins	2	0	2	100.00

User Defined Bins for cp_pwrite

Bins

NAME	COUNT	AT LEAST
read	54	1
write	46	1

4. Verification Results

Group : $unit::transaction#(32)::cov_slv_apb

SCORE	WEIGHT	GOAL	AT LEAST	AUTO BIN MAX	PRINT MISSING
100.00	1	100	1	64	64

Summary for Group $unit::transaction#(32)::cov_slv_apb

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
Variables	8	0	8	100.00
Crosses	6	0	6	100.00

Variables for Group $unit::transaction#(32)::cov_slv_apb

VARIABLE	EXPECTED	UNCOVERED	COVERED	PERCENT	GOAL	WEIGHT	AT LEAST
cp_paddr	6	0	6	100.00	100	1	
cp_pselx	2	0	2	100.00	100	1	

Crosses for Group $unit::transaction#(32)::cov_slv_apb

CROSS	EXPECTED	UNCOVERED	COVERED	PERCENT	GOAL	WEIGHT	AT LEAST
cross_paddr_pselx	6	0	6	100.00	100	1	

Variable : cp_paddr 〉 Variable : cp_pselx 〉 Cross : cross_paddr_pselx 〉

Summary for Variable cp_paddr

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
User Defined Bins	6	0	6	100.00

User Defined Bins for cp_paddr

Bins

NAME	COUNT	AT LEAST
waiting_addr_slv1	15	1
waiting_addr_slv0	26	1
addr_slave1	10	1
addr_error	10	1
rand_addr_in_range	49	1
fixed_addr	25	1

Summary for Variable cp_pselx

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
User Defined Bins	2	0	2	100.00

User Defined Bins for cp_pselx

Bins

NAME	COUNT	AT LEAST
slave_1	33	1
slave_0	91	1

Summary for Cross cross_paddr_pselx

Samples crossed: cp_paddr cp_pselx

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT	MISSING
TOTAL	6	0	6	100.00	
Automatically Generated Cross Bins	2	0	2	100.00	
User Defined Cross Bins	4	0	4	100.00	

Automatically Generated Cross Bins for cross_paddr_pselx

Bins

cp_paddr	cp_pselx	COUNT	AT LEAST
rand_addr_in_range	slave_1	8	1
rand_addr_in_range	slave_0	41	1

User Defined Cross Bins for cross_paddr_pselx

Excluded/Illegal bins

NAME	COUNT	STATUS
false_addr_slv0	0	Excluded
false_addr_slv1	0	Excluded
error_addr0	0	Excluded
error_addr1	0	Excluded

Covered bins

NAME	COUNT	AT LEAST
addr_slv0	25	1
addr_slv1	10	1
waiting_slv0	26	1
waiting_slv1	15	1

Thanks for watching

Digital VLSI Design

Actor: Hoang Bao Long